



BÁO CÁO TIỂU LUẬN KHÓA LUẬN

MÔN HỌC: GIẢI THUẬT NÂNG CAO

ĐỀ TÀI CHUYÊN SÂU:

**NGHIÊN CỨU VỀ THUẬT TOÁN ZERO-DCE:
TĂNG CƯỜNG ẢNH THIẾU SÁNG BẰNG PHƯƠNG PHÁP
ƯỚC LƯỢNG ĐƯỜNG CONG KHÔNG THAM CHIỀU VÀ CÁC
THỂ NGHIỆM**

Sinh viên thực hiện:	Đỗ Hữu Khang
Lớp:	KHOA HỌC MÁY TÍNH
Mã số sinh viên:	2580101060
Giảng viên hướng dẫn:	Trần Lê Anh

Mục lục

1	Giới thiệu chi tiết và Tầm quan trọng của Đề tài	5
1.1	Đặt vấn đề và Tầm quan trọng của Đề tài	5
1.2	Cơ chế sinh lý thị giác và vật lý cảm biến	5
1.3	Ứng dụng thực tế và Bố cục Báo cáo	5
2	Các công trình liên quan và Phân tích Lịch sử	7
2.1	Phương pháp xử lý ảnh trong miền không gian (Spatial Domain Methods)	7
2.2	Lý thuyết Retinex và các phương pháp lọc tần số	7
2.3	Các phương pháp dựa trên Học sâu (Deep Learning Methods)	8
3	Kiến trúc Hệ thống và Cơ sở Lý thuyết của Zero-DCE	9
3.1	Thiết kế Đường cong Tăng cường Ánh sáng phi tuyến thích ứng (LE-curve)	9
3.1.1	Phương trình toán học cơ bản và Ý nghĩa vật lý	9
3.1.2	Chứng minh toán học chặt chẽ về tính đơn điệu và tính bị chặn của LE-curve	10
3.1.3	Thiết kế đường cong lặp bậc cao thích ứng (Higher-Order Curves)	11
3.1.4	Ví dụ minh họa chi tiết quá trình tăng cường điểm ảnh	11
3.2	Kiến trúc mạng ước lượng tham số DCE-Net	13
3.2.1	Cấu trúc chi tiết các lớp mạng và Skip Connections	13
4	Phân tích Chuyên sâu về Bộ bốn Hàm Mất mát Không Tham chiều	15
4.1	Hàm mất mát nhất quán không gian (Spatial Consistency Loss - L_{spa}) . .	15
4.2	Hàm mất mát kiểm soát phơi sáng (Exposure Control Loss - L_{exp})	15
4.3	Hàm mất mát hằng số màu sắc (Color Constancy Loss - L_{col})	16
4.4	Hàm mất mát độ mượt ánh sáng (Illumination Smoothness Loss - L_{tv}) . .	17
4.5	Tỷ lệ kết hợp các hàm mất mát	17
5	Quy trình Huấn luyện Mô hình	19
5.1	Chuẩn bị dữ liệu (Data Preparation)	19
5.2	Quá trình lan truyền xuôi (Forward Pass)	19
5.3	Đánh giá chất lượng bằng Bộ bốn Hàm mất mát không tham chiều	19
5.4	Hiện thực hóa Quy trình Huấn luyện bằng Mã nguồn PyTorch	20
5.4.1	Cấu hình các siêu tham số huấn luyện (Config Class)	20
5.4.2	Bộ nạp dữ liệu tùy chỉnh (LowLightDataset & DataLoader)	21
5.4.3	Hiện thực hóa kiến trúc mạng DCE-Net trong PyTorch	22
5.4.4	Hiện thực hóa 4 Hàm Mất Mát Không Tham Chiều	24
5.4.5	Vòng lặp Huấn luyện Hệ thống (Training Loop)	27

5.4.6	Xuất mô hình sang định dạng ONNX phục vụ triển khai thực tế .	29
6	Kết quả Thực nghiệm, Bảng số liệu và Thảo luận chuyên sâu	31
6.1	Đánh giá Định lượng trên các tập dữ liệu tiêu chuẩn	31
6.2	Đánh giá Định tính và Trực quan hình ảnh	32
6.3	Tốc độ xử lý và Khả năng đáp ứng thời gian thực	32
6.4	Phân tích Đạo hàm và Khả năng hội tụ toán học của mô hình	33
6.5	Khảo sát các trường hợp thất bại khách quan (Failure Cases Analysis) . .	33
7	Ứng dụng Thực tế và Các bài toán Hạ nguồn	35
7.1	Tác động tích cực đến các bài toán thị giác máy tính hạ nguồn (Down-stream Tasks)	35
7.2	Triển khai tối ưu hóa thời gian thực trên Thiết bị biên (Edge Devices) . .	35
8	Đề xuất Cải tiến và Hướng phát triển trong Tương lai	37
8.1	Tổng kết những đóng góp của báo cáo nghiên cứu	37
8.2	Đề xuất mô hình cải tiến thế hệ mới: Zero-DCE++	37
8.3	Đề xuất tối ưu hóa toán học cho các điểm ảnh đã đủ sáng	38

Danh sách bảng

1	Bảng so sánh tổng quan các nhóm phương pháp tăng cường ảnh thiếu sáng	8
2	Đánh giá định lượng (PSNR/SSIM trên tập LOL; chỉ số NIQE trung bình trên 4 tập dữ liệu không tham chiếu LIME, MEF, DICM, VV)	31
3	So sánh tốc độ xử lý (Frame Per Second - FPS) trên các phần cứng khác nhau	32

Tóm tắt nội dung

Tăng cường hình ảnh trong điều kiện thiếu sáng (Low-Light Image Enhancement - LLIE) là một thách thức quan trọng và lâu đời trong lĩnh vực thị giác máy tính cũng như xử lý ảnh kỹ thuật số. Các phương pháp học sâu truyền thống thường dựa trên việc huấn luyện có giám sát (supervised learning) với các cặp hình ảnh thiếu sáng và ảnh chuẩn (ground-truth) có độ phơi sáng hoàn hảo. Tuy nhiên, trong thực tế, việc thu thập các cặp dữ liệu tính này vô cùng khó khăn, tốn kém và thường dễ bị lệch điểm ảnh do các chuyển động rung lắc nhẹ của camera hoặc sự thay đổi của vật thể trong khung hình.

Bài báo cáo nghiên cứu này trình bày chi tiết và toàn diện về thuật toán **Zero-DCE** (Zero-Reference Deep Curve Estimation), một phương pháp tiếp cận không giám sát đột phá giải quyết bài toán tăng cường ánh sáng mà không cần bất kỳ hình ảnh tham chiếu nào trong quá trình huấn luyện. Triết lý thiết kế của Zero-DCE coi bài toán tăng cường phơi sáng là bài toán ước lượng các đường cong điều chỉnh ánh sáng đặc thù cho từng điểm ảnh (pixel-wise light enhancement curves - LE-curves) thông qua một mạng thần kinh tích chập siêu nhẹ mang tên **DCE-Net**.

Trong báo cáo này, chúng tôi phân tích sâu sắc cơ sở lý thuyết toán học của các đường cong tăng cường, kiến trúc mạng FCN đối xứng, và bộ bốn hàm mất mát không tham chiếu (nhất quán không gian, kiểm soát phơi sáng, hằng số màu sắc và độ mượt ánh sáng).

Kết quả thực nghiệm cho thấy Zero-DCE đạt được hiệu suất vượt trội về cả chất lượng cảm quan lẫn tốc độ xử lý (đạt tới 500 FPS trên GPU phổ thông), mở ra triển vọng ứng dụng thời gian thực trên các thiết bị di động và các hệ thống biên tự hành.

1 Giới thiệu chi tiết và Tầm quan trọng của Đề tài

1.1 Đặt vấn đề và Tầm quan trọng của Đề tài

Trong kỷ nguyên số, thị giác máy tính và trí tuệ nhân tạo đã trở thành những trụ cột cốt lõi của cuộc Cách mạng Công nghiệp 4.0. Các hệ thống như camera giám sát thông minh, xe tự hành, máy bay không người lái, y tế và nhiếp ảnh điện toán đều phụ thuộc hoàn toàn vào chất lượng dữ liệu hình ảnh đầu vào. Tuy nhiên, việc thu nhận hình ảnh trong điều kiện thiếu sáng (underexposure) luôn là rào cản kỹ thuật lớn do giới hạn vật lý của cảm biến quang học, gây ra các hiện tượng suy giảm nghiêm trọng: độ tương phản cực thấp làm mất ranh giới cấu trúc vật thể; nhiễu hạt (noise) và nhiễu màu mạnh do tăng ISO quá mức; sai lệch màu sắc (color distortion) do tỷ lệ tín hiệu trên nhiễu (SNR) thấp.

Do đó, việc nghiên cứu các giải pháp tăng cường ảnh thiếu sáng (Low-Light Image Enhancement - LLIE) đóng vai trò sống còn trong việc nâng cao độ chính xác của các tác vụ AI hạ nguồn như phát hiện vật thể, phân vùng ngữ cảnh và nhận diện khuôn mặt.

1.2 Cơ chế sinh lý thị giác và vật lý cảm biến

Để giải quyết hiệu quả bài toán LLIE, cần thấu hiểu sự khác biệt giữa thị giác con người và cảm biến kỹ thuật số:

- **Thị giác bóng tối ở người (Scotopic Vision):** Sử dụng tế bào que võng mạc kết hợp với cơ chế thích nghi phi tuyến của não bộ để nhận biết hình khối mượt mà trong bóng đêm mà không bị nhiễu hạt hay mất dải màu.
- **Cơ chế nhiễu cảm biến vật lý (CMOS/CCD):** Khi thiếu photon, các cảm biến phải đối mặt với các nguồn nhiễu ngẫu nhiên phức tạp bao gồm nhiễu phát xạ photon (Photon Shot Noise) tuân theo phân phối Poisson, nhiễu tối nhiệt (Dark Current Noise) do nhiệt độ kích thích electron tự do, và nhiễu đọc mạch điện (Readout Noise) trong quá trình ADC.

Các phương pháp phần mềm bắt buộc phải tìm cách nâng sáng tín hiệu hữu ích đồng thời ức chế các nguồn nhiễu vật lý này.

1.3 Ứng dụng thực tế và Bổ cục Báo cáo

Thuật toán tăng cường ảnh thiếu sáng có phạm vi ứng dụng thực tế vô cùng rộng lớn: từ hệ thống hỗ trợ lái xe nâng cao (ADAS) giúp xe tự hành di chuyển an toàn vào ban đêm; camera giám sát an ninh nhận diện khuôn mặt nghi phạm; thiết bị bay không người lái trong tìm kiếm cứu nạn; cho đến nội soi y tế giúp bác sĩ chẩn đoán mô tổn thương.

Để trình bày một cách hệ thống, báo cáo nghiên cứu này được chia thành các chương cốt lõi: Chương 1 giới thiệu tầm quan trọng và bối cảnh đề tài; Chương 2 phân tích lịch sử các phương pháp từ truyền thống đến học sâu; Chương 3 trình bày lý thuyết đường cong LE-curve phi tuyến; Chương 4 đi sâu vào cấu trúc bộ 4 hàm loss không tham chiếu; Chương 5 giới thiệu chi tiết mã nguồn PyTorch và quy trình huấn luyện; Chương 6 phân tích các kết quả định lượng, định tính và tốc độ xử lý; và Chương 7 đề xuất các hướng cải tiến tương lai bao gồm mô hình Zero-DCE++.

2 Các công trình liên quan và Phân tích Lịch sử

2.1 Phương pháp xử lý ảnh trong miền không gian (Spatial Domain Methods)

Các phương pháp truyền thống tiếp cận bài toán tăng cường ảnh thiếu sáng bằng cách biến đổi trực tiếp giá trị phân phối điểm ảnh trên ma trận ảnh gốc. Hai kỹ thuật kinh điển nhất là Cân bằng lược đồ xám (Histogram Equalization) và Hiệu chỉnh Gamma (Gamma Correction).

- **Cân bằng lược đồ xám toàn cục (Global Histogram Equalization - GHE):** Dàn trải phân phối mức xám của hình ảnh để đạt được lược đồ tần suất phân bố đều, mở rộng dải động. Hàm biến đổi CDF được định nghĩa là $s = T(r) = \int_0^r p_r(w)dw$. GHE tính toán rất nhanh nhưng dễ gây ra hiện tượng tăng sáng quá mức (over-enhancement) ở các vùng vốn đã đủ sáng và làm mất chi tiết.
- **Cân bằng lược đồ xám thích ứng giới hạn tương phản (CLAHE):** CLAHE chia hình ảnh thành các ô nhỏ (tiles) không chồng chéo, cân bằng lược đồ xám độc lập cho từng ô và giới hạn mức cắt (clipping limit) để tránh phóng đại nhiễu. Phép nội suy song tuyến (bilinear interpolation) được sử dụng để làm mịn giá trị điểm ảnh chuyển tiếp giữa các ô:

$$I_{\text{new}}(x, y) = (1 - a)(1 - b)I_A + a(1 - b)I_B + (1 - a)bI_C + abI_D \quad (1)$$

2.2 Lý thuyết Retinex và các phương pháp lọc tần số

Lý thuyết Retinex do Edwin H. Land đề xuất dựa trên giả định rằng hình ảnh cảm nhận được $S(x, y)$ là tích số của hai thành phần: ánh sáng môi trường chiếu vào vật thể (Illumination $I(x, y)$) và tính chất phản xạ tự thân của vật thể (Reflectance $R(x, y)$):

$$S(x, y) = I(x, y) \times R(x, y) \implies \log S(x, y) = \log I(x, y) + \log R(x, y) \quad (2)$$

Mục tiêu của các phương pháp Retinex là loại bỏ $I(x, y)$ để khôi phục $R(x, y)$ sắc nét và chân thực:

- **Retinex đơn tỷ lệ (SSR) và đa tỷ lệ (MSR):** SSR ước lượng bản đồ chiếu sáng $I(x, y)$ bằng cách lọc thông thấp ảnh gốc qua bộ lọc Gauss $G(x, y)$ với một hệ số tỷ lệ σ . MSR cải tiến SSR bằng cách kết hợp tuyến tính nhiều bộ lọc Gauss với các tỷ lệ khác nhau ($\sigma_1, \sigma_2, \dots, \sigma_N$). Để bù đắp việc mất độ bão hòa màu, MSRCR tích hợp thêm hàm phục hồi màu sắc $C_c(x, y)$ cho mỗi kênh màu $c \in \{R, G, B\}$.

- **Phương pháp ước lượng bản đồ chiếu sáng tối ưu hóa (LIME):** LIME ước lượng bản đồ chiếu sáng thô ban đầu bằng cách tìm cường độ pixel cực đại trên ba kênh màu, sau đó giải bài toán tối ưu hóa có số hạng phạt độ mịn không gian dựa trên gradient ảnh để tinh chỉnh bản đồ chiếu sáng. LIME cho hiệu năng khôi phục biên rất sắc nét nhưng dễ làm phóng đại nhiễu hạt ở các vùng quá tối.

2.3 Các phương pháp dựa trên Học sâu (Deep Learning Methods)

Sự phát triển của học sâu (Deep Learning) đã cách mạng hóa bài toán LLIE. Các phương pháp học sâu được chia thành ba nhánh chính:

- **Học sâu có giám sát (Supervised DL):** Các kiến trúc tiêu biểu như LLNet, RetinexNet hay KinD sử dụng mạng nơ-ron để học hàm ánh xạ phi tuyến từ ảnh tối sang ảnh sáng. Tuy nhiên, các mô hình này phụ thuộc nặng nề vào các cặp dữ liệu tối-sáng nhân tạo (paired data), dẫn đến khả năng tổng quát hóa kém và dễ bị lệch phân phối nhiều khi áp dụng vào ảnh thực tế.
- **Học sâu không giám sát dựa trên mạng GAN (Unsupervised GANs):** Như EnlightenGAN sử dụng cấu trúc Generator-Discriminator để tự học ánh sáng từ hai tập dữ liệu không ghép cặp (unpaired data). Mặc dù giải quyết được vấn đề dữ liệu cặp, GAN rất khó huấn luyện do hiện tượng sụp mode (mode collapse), cấu trúc công kênh khó chạy thời gian thực và đôi khi tự sinh các chi tiết giả (artifacts).
- **Học sâu không tham chiếu (Zero-Reference DL):** Tiêu biểu là **Zero-DCE** [1]. Mô hình này loại bỏ hoàn toàn sự phụ thuộc vào dữ liệu cặp hoặc bộ phân biệt đối kháng. Nó sử dụng một mạng tích chập siêu nhẹ (DCE-Net) để ước lượng các bản đồ tham số đường cong điều chỉnh ánh sáng phi tuyến, hội tụ ổn định thông qua bộ bốn hàm mất mát không tham chiếu xây dựng từ các tiên nghiệm vật lý, mang lại tốc độ xử lý vượt trội và khả năng tổng quát hóa xuất sắc.

Bảng 1: Bảng so sánh tổng quan các nhóm phương pháp tăng cường ảnh thiếu sáng

Nhóm phương pháp	Yêu cầu dữ liệu	Tài nguyên tính toán	Độ ổn định huấn luyện	Hiện tượng Artifacts
Spatial (HE/CLAHE)	Không cần huấn luyện	Cực thấp	Không áp dụng	Rất cao (cháy sáng, nhiễu)
Retinex (MSRCR/LIME)	Không cần huấn luyện	Thấp đến Trung bình	Không áp dụng	Trung bình (quầng sáng halo)
Supervised DL	Cần ảnh cặp tối-sáng	Cao (khi huấn luyện)	Cao	Thấp (nhưng kém tổng quát)
Unsupervised GAN	Ảnh không cặp tối/sáng	Cực cao (huấn luyện)	Thấp (dễ sụp mode)	Có thể xuất hiện chi tiết giả
Zero-DCE (Đề xuất)	Chỉ cần ảnh tối	Thấp đến Cực thấp	Tuyệt đối ổn định	Hầu như không có

3 Kiến trúc Hệ thống và Cơ sở Lý thuyết của Zero-DCE

3.1 Thiết kế Đường cong Tăng cường Ánh sáng phi tuyến thích ứng (LE-curve)

Thay vì đi theo lối mòn của các mạng học sâu truyền thống là cố gắng huấn luyện mạng nơ-ron học một hàm ánh xạ phi tuyến cực kỳ phức tạp từ không gian pixel tối sang không gian pixel sáng (để dẫn đến việc overfitting và sinh ra nhiễu), Zero-DCE sử dụng mạng nơ-ron để ước lượng tham số của một họ đường cong toán học đơn giản nhưng mạnh mẽ. Đường cong này được lấy cảm hứng từ tính năng "Curves" trong các công cụ chỉnh sửa ảnh chuyên nghiệp như Adobe Photoshop hay Lightroom.

3.1.1 Phương trình toán học cơ bản và Ý nghĩa vật lý

Hàm tăng cường ánh sáng cơ bản (Light Enhancement Curve - LE-curve) cấp 1 được định nghĩa thông qua phương trình bậc hai sau:

$$LE(I(x); \alpha) = I(x) + \alpha I(x)(1 - I(x)) \quad (3)$$

Trong đó:

- $I(x)$ là giá trị độ sáng của một điểm ảnh tại vị trí không gian x , được chuẩn hóa về đoạn $[0, 1]$ từ không gian 8-bit gốc:

$$I(x) = \frac{x_{\text{RGB}}}{255} \quad (4)$$

- $\alpha \in [-1, 1]$ là tham số điều khiển độ cong phi tuyến của đường cong, được dự đoán độc lập cho từng điểm ảnh bởi mạng DCE-Net.

Ý nghĩa vật lý của thành phần hiệu chỉnh $\alpha I(x)(1 - I(x))$ là cực kỳ tinh tế:

- Khi $\alpha > 0$, đường cong lồi lên phía trên, làm tăng giá trị phơi sáng của điểm ảnh đầu ra.
- Khi $\alpha < 0$, đường cong lõm xuống dưới, làm giảm độ sáng của điểm ảnh (hữu ích để làm dịu các vùng quá chói).
- Tại các điểm giới hạn biên tuyệt đối $I(x) = 0$ (tối đen hoàn toàn) và $I(x) = 1$ (sáng trắng hoàn toàn), thành phần $I(x)(1 - I(x))$ tự động triệt tiêu về 0. Do đó, phương trình đảm bảo giữ nguyên $LE(0; \alpha) = 0$ và $LE(1; \alpha) = 1$ trong mọi trường hợp. Điều này ngăn chặn hoàn toàn hiện tượng mất chi tiết do cắt cụt biên (clipping artifacts) thường gặp ở các phương pháp truyền thống.

3.1.2 Chứng minh toán học chặt chẽ về tính đơn điệu và tính bị chặn của LE-curve

Một yêu cầu bắt buộc đối với mọi hàm biến đổi ảnh là phải bảo toàn thứ tự sáng tối của các điểm ảnh gốc (tính đơn điệu tăng). Nếu điểm ảnh A tối hơn điểm ảnh B trong ảnh gốc, thì sau khi tăng sáng, điểm ảnh A vẫn phải tối hơn điểm ảnh B để tránh hiện tượng đảo ngược tương phản (contrast reversal).

Chúng tôi đưa ra chứng minh toán học chặt chẽ dưới đây:

Định lý: Hàm số $f(x) = x + \alpha x(1 - x)$ là hàm số đơn điệu tăng và bị chặn trong đoạn $x \in [0, 1]$ khi và chỉ khi $\alpha \in [-1, 1]$.

Chứng minh: Khai triển hàm số $f(x)$ ta được:

$$f(x) = (1 + \alpha)x - \alpha x^2 \quad (5)$$

Lấy đạo hàm bậc nhất của $f(x)$ theo biến số x :

$$f'(x) = 1 + \alpha - 2\alpha x \quad (6)$$

Để hàm số $f(x)$ đơn điệu tăng trên toàn bộ đoạn đóng $[0, 1]$, ta cần điều kiện đạo hàm không âm trên đoạn này:

$$f'(x) \geq 0 \quad \forall x \in [0, 1] \quad (7)$$

Vì $f'(x)$ là một hàm bậc nhất (hàm tuyến tính) đối với biến số x , giá trị nhỏ nhất của nó trên đoạn $[0, 1]$ bắt buộc phải nằm tại một trong hai đầu mút của đoạn đóng, tức là tại $x = 0$ hoặc $x = 1$. Do đó, điều kiện $f'(x) \geq 0$ với mọi $x \in [0, 1]$ tương đương với hệ bất phương trình sau:

$$\begin{cases} f'(0) \geq 0 \\ f'(1) \geq 0 \end{cases} \quad (8)$$

Thay các giá trị đầu mút vào phương trình đạo hàm:

- Tại $x = 0$: $f'(0) = 1 + \alpha - 2\alpha(0) = 1 + \alpha \geq 0 \implies \alpha \geq -1$
- Tại $x = 1$: $f'(1) = 1 + \alpha - 2\alpha(1) = 1 - \alpha \geq 0 \implies \alpha \leq 1$

Kết hợp hai bất phương trình trên, ta thu được điều kiện cần và đủ cho tham số α là:

$$-1 \leq \alpha \leq 1 \quad (9)$$

Đồng thời, ta kiểm tra tính bị chặn của hàm số. Với $\alpha \in [-1, 1]$ và $x \in [0, 1]$:

- Giá trị biên dưới: $f(0) = 0$
- Giá trị biên trên: $f(1) = 1$

- Vì $f(x)$ liên tục và đơn điệu tăng trên $[0, 1]$, tập giá trị của nó là $f([0, 1]) = [f(0), f(1)] = [0, 1]$.

Như vậy, hàm số hoàn toàn bị chặn trong đoạn đóng $[0, 1]$, không bao giờ vượt ngưỡng gây cháy sáng hoặc âm màu. Định lý được chứng minh hoàn toàn.

3.1.3 Thiết kế đường cong lặp bậc cao thích ứng (Higher-Order Curves)

Mặc dù đường cong cấp 1 bảo toàn biên tốt, một phương trình bậc hai đơn lẻ không đủ khả năng điều chỉnh linh hoạt cho các bức ảnh có mức độ thiếu sáng quá sâu. Để tăng cường khả năng thích nghi phi tuyến, Zero-DCE thực hiện áp dụng lặp lại đường cong cơ bản nhiều lần:

$$I_k(x) = LE(I_{k-1}(x); \alpha_k) = I_{k-1}(x) + \alpha_k I_{k-1}(x)(1 - I_{k-1}(x)) \quad (10)$$

với $k \in \{1, 2, \dots, N\}$ là số bước lặp. Tại mỗi bước lặp k , mô hình sử dụng một bản đồ tham số α_k riêng biệt.

Nếu ta lặp lại N lần, phương trình cuối cùng sẽ có bậc toán học là 2^N . Trong cấu hình mặc định của Zero-DCE, tác giả thiết lập $N = 8$. Hàm biến đổi lúc này là một đa thức bậc 256. Bậc tự do cực cao này cho phép mô hình khôi phục chi tiết ánh sáng vô cùng tinh tế ở các vùng cực tối mà không làm ảnh hưởng đến vùng sáng có sẵn.

3.1.4 Ví dụ minh họa chi tiết quá trình tăng cường điểm ảnh

Để hiểu rõ hơn về cách LE-curve hoạt động và bảo toàn các vùng sáng/tối tuyệt đối, chúng ta hãy xem xét một ví dụ thực tế với 3 điểm ảnh ban đầu có giá trị kênh màu RGB lần lượt là Đen (0, 0, 0), Trắng (255, 255, 255), và Xám tối (51, 51, 51).

Trước khi đưa vào công thức tăng cường, các giá trị này được chuẩn hóa về khoảng $[0, 1]$ bằng cách chia giá trị của từng kênh màu cho 255.

Vì đây là các điểm ảnh đơn sắc hoặc xám (có giá trị của ba kênh màu R, G, B trùng nhau), ta có thể đại diện giá trị tính toán thông qua một kênh duy nhất nhằm đơn giản hóa minh họa:

- **Đen** ($R = 0, G = 0, B = 0$): Do điểm ảnh đen hoàn toàn có giá trị cường độ trên cả ba kênh bằng 0, ta có giá trị đầu vào chuẩn hóa tương ứng là $I_0 = \frac{0}{255} = 0.0$.
- **Trắng** ($R = 255, G = 255, B = 255$): Do điểm ảnh trắng hoàn toàn đạt mức cường độ tối đa bằng 255 trên cả ba kênh, ta thu được giá trị chuẩn hóa là $I_0 = \frac{255}{255} = 1.0$.
- **Xám tối** ($R = 51, G = 51, B = 51$): Do điểm ảnh xám tối có giá trị kênh màu bằng 51, ta thu được giá trị chuẩn hóa là $I_0 = \frac{51}{255} = 0.2$.

Lưu ý đối với ảnh màu: Đối với các điểm ảnh có màu sắc bất kỳ (khi các kênh R, G, B có giá trị khác nhau), ví dụ điểm ảnh có mã màu RGB là (30, 24, 60), thuật toán sẽ chuẩn hóa độc lập từng kênh màu trước khi đưa vào tính toán:

- Kênh Red ($R = 30$): $I_{0,R} = \frac{30}{255} \approx 0.118$
- Kênh Green ($G = 24$): $I_{0,G} = \frac{24}{255} \approx 0.094$
- Kênh Blue ($B = 60$): $I_{0,B} = \frac{60}{255} \approx 0.235$

Sau đó, mỗi kênh $I_{0,R}$, $I_{0,G}$, $I_{0,B}$ sẽ được tăng cường song song và độc lập bằng các tham số α_R , α_G , α_B tương ứng được dự đoán bởi mạng thần kinh.

Giả sử mạng DCE-Net dự đoán tham số $\alpha = 0.5$ (hệ số kéo sáng) cho cả 3 điểm ảnh đơn sắc ở trên và chúng ta thực hiện tính toán qua 3 vòng lặp ($n = 3$):

- **Điểm ảnh Đen** ($I_0 = 0.0$):
 - Lần lặp 1: $I_1 = 0.0 + 0.5 \times 0.0 \times (1 - 0.0) = 0.0$
 - Kết quả sau 3 vòng lặp vẫn là 0.0. Tính chất này cho thấy thuật toán không làm "xám hóa" các vùng tối đen hoàn toàn trong ảnh gốc, giúp bảo toàn độ tương phản tự nhiên của vật thể.
- **Điểm ảnh Trắng** ($I_0 = 1.0$):
 - Lần lặp 1: $I_1 = 1.0 + 0.5 \times 1.0 \times (1 - 1.0) = 1.0 + 0 = 1.0$
 - Kết quả sau 3 vòng lặp vẫn là 1.0. Nhờ thiết kế toán học khéo léo, hàm sẽ triệt tiêu phần tăng cường khi $I(x)$ tiến tới 1, giúp thuật toán tự động tránh hiện tượng lóa sáng (over-exposure) ở các nguồn sáng.
- **Điểm ảnh Xám tối** ($I_0 = 0.2$):
 - Lần lặp 1: $I_1 = 0.2 + 0.5 \times 0.2 \times (1 - 0.2) = 0.2 + 0.08 = 0.28$
 - Lần lặp 2: $I_2 = 0.28 + 0.5 \times 0.28 \times (1 - 0.28) = 0.28 + 0.5 \times 0.28 \times 0.72 = 0.28 + 0.1008 = 0.3808$
 - Lần lặp 3: $I_3 = 0.3808 + 0.5 \times 0.3808 \times (1 - 0.3808) \approx 0.3808 + 0.5 \times 0.3808 \times 0.6192 \approx 0.3808 + 0.1179 \approx 0.4987$

Sau khi thu được giá trị cường độ chuẩn hóa ở vòng lặp cuối cùng (ở đây là I_3), thuật toán thực hiện khôi phục điểm ảnh về không gian màu 8-bit ban đầu ($[0, 255]$) bằng cách nhân với 255 và làm tròn đến số nguyên gần nhất:

$$x_{\text{RGB, final}} = \text{round}(I_n(x) \times 255) \quad (11)$$

Áp dụng công thức khôi phục này cho 3 điểm ảnh trong ví dụ trên:

- **Điểm ảnh Đen:** $x_{\text{RGB}, \text{final}} = \text{round}(0.0 \times 255) = 0 \rightarrow$ Điểm ảnh khôi phục là Đen $(0, 0, 0)$.
- **Điểm ảnh Trắng:** $x_{\text{RGB}, \text{final}} = \text{round}(1.0 \times 255) = 255 \rightarrow$ Điểm ảnh khôi phục là Trắng $(255, 255, 255)$.
- **Điểm ảnh Xám tối:** $x_{\text{RGB}, \text{final}} = \text{round}(0.4987 \times 255) = \text{round}(127.1685) \approx 127 \rightarrow$ Điểm ảnh khôi phục là Xám trung tính $(127, 127, 127)$.

Thông qua ví dụ toán học trên, ta thấy điểm ảnh xám tối ban đầu từ mức màu $(51, 51, 51)$ đã được tăng cường độ sáng lên thành màu xám sáng hơn nhiều là $(127, 127, 127)$ một cách mượt mà và tiệm tiến qua từng bước. Trong khi đó, các giới hạn đen/trắng hai đầu không hề bị phá vỡ hay bị cắt cụt (clipping).

Trong cấu hình thực tế của mô hình gốc, Zero-DCE lặp lại $N = 8$ lần với các giá trị α_k khác nhau được dự đoán độc lập tại mỗi vòng lặp bởi mạng DCE-Net, tạo ra quỹ đạo kéo sáng cực kỳ tinh tế và phức tạp, thích nghi linh hoạt với từng vùng sáng tối khác nhau trên bức ảnh.

3.2 Kiến trúc mạng ước lượng tham số DCE-Net

Để ước lượng các bản đồ tham số đường cong $\mathcal{A} = \{\alpha_1, \alpha_2, \dots, \alpha_N\}$, Zero-DCE sử dụng một mạng nơ-ron tích chập hoàn toàn (Fully Convolutional Network - FCN) gọn nhẹ mang tên **DCE-Net**.

3.2.1 Cấu trúc chi tiết các lớp mạng và Skip Connections

DCE-Net được thiết kế với sự đơn giản tối đa để tối ưu hiệu suất tính toán. Mạng gồm 7 lớp tích chập đối xứng và không sử dụng bất kỳ lớp Pooling hay Fully Connected nào nhằm bảo toàn nguyên vẹn độ phân giải không gian của ảnh đầu vào.

- **Lớp 1 đến Lớp 6:** Mỗi lớp gồm 32 bộ lọc (filters) kích thước 3×3 , bước dịch chuyển (stride) bằng 1, kết hợp đệm rìa (padding) bằng 1 để giữ nguyên kích thước ma trận ảnh. Hàm kích hoạt được chọn là ReLU.
- **Lớp 7 (Lớp đầu ra):** Gồm 24 bộ lọc kích thước 3×3 . Tại sao lại là 24 bộ lọc? Bởi vì mô hình cần dự đoán bản đồ tham số α cho 3 kênh màu riêng biệt (R, G, B) qua $N = 8$ vòng lặp ($3 \text{ kênh} \times 8 \text{ vòng lặp} = 24 \text{ bản đồ tham số}$). Hàm kích hoạt của lớp 7 là Tanh, giúp ép chặt đầu ra α về đoạn $[-1, 1]$, thỏa mãn chính xác điều kiện đơn điệu tăng đã được chứng minh ở mục trước.

Để giải quyết vấn đề mất mát thông tin ngữ cảnh không gian khi đi qua nhiều lớp tích chập sâu, DCE-Net áp dụng cơ chế kết nối tắt đối xứng (Symmetric Skip Connections)

tương tự như kiến trúc U-Net. Đầu ra của các lớp được nối lại với nhau dọc theo chiều kênh (channel concatenation):

- Đầu vào lớp 6 là kết hợp của đầu ra lớp 5 và lớp 2.
- Đầu vào lớp 5 là kết hợp của đầu ra lớp 4 và lớp 3.
- Việc kết nối này giúp mạng giữ lại các đặc trưng cấp thấp (độ chi tiết biên, cấu trúc texture) kết hợp với đặc trưng ngữ cảnh cấp cao.

4 Phân tích Chuyên sâu về Bộ bốn Hàm Mất mát Không Tham chiếu

Trọng tâm tạo nên sức mạnh "không tham chiếu"(zero-reference) của Zero-DCE nằm ở việc thiết kế bộ bốn hàm mất mát tự giám sát. Các hàm này đóng vai trò người giám khảo tự đánh giá chất lượng bức ảnh đầu ra dựa trên các thuộc tính vật lý và thẩm mỹ học thị giác, thay thế hoàn toàn cho ảnh chuẩn ground-truth.

4.1 Hàm mất mát nhất quán không gian (Spatial Consistency Loss - L_{spa})

Nhiệm vụ của L_{spa} là bảo toàn cấu trúc ranh giới hình học và độ tương phản tương đối giữa các vùng lân cận của bức ảnh. Khi tăng sáng, mô hình không được phép làm mịn phẳng các chi tiết biên hoặc thay đổi thứ tự sáng tối cục bộ.

Phương trình toán học của L_{spa} được định nghĩa là:

$$L_{spa} = \frac{1}{K} \sum_{i=1}^K \sum_{j \in \Omega(i)} (|(Y_i - Y_j)| - |(I_i - I_j)|)^2 \quad (12)$$

Trong đó:

- I là ảnh đầu vào thiếu sáng, Y là ảnh đầu ra sau khi được tăng cường.
- i là chỉ số của vùng cục bộ đang xét. K là tổng số vùng cục bộ trên toàn bức ảnh.
- $\Omega(i)$ tập hợp bốn vùng lân cận trực tiếp của vùng i (bao gồm: Trên, Dưới, Trái, Phải).
- Giá trị chênh lệch cục bộ $|Y_i - Y_j|$ và $|I_i - I_j|$ thực chất là các gradient không gian. Bằng cách bắt gradient của ảnh đầu ra phải bám sát gradient của ảnh đầu vào, mô hình giữ vững cấu trúc sắc nét của các cạnh vật thể. Kích thước của mỗi vùng cục bộ được thiết lập mặc định là 4×4 pixel để đạt được sự cân bằng tối ưu giữa việc giữ cấu trúc vi mô và vĩ mô.

4.2 Hàm mất mát kiểm soát phơi sáng (Exposure Control Loss - L_{exp})

Để ngăn ngừa tình trạng ảnh đầu ra bị cháy sáng (over-exposure) hoặc vẫn chưa đủ sáng (under-exposure), hàm mất mát L_{exp} đo lường mức độ sai lệch của độ sáng trung bình trong các vùng cục bộ so với một giá trị độ sáng mục tiêu lý tưởng E .

Công thức tính toán cụ thể:

$$L_{exp} = \frac{1}{M} \sum_{k=1}^M |\bar{Y}_k - E| \quad (13)$$

Trong đó:

- M là số lượng ô cửa sổ cục bộ kích thước 16×16 pixel không chồng chéo được phân hoạch trên bức ảnh.
- $\bar{Y}_k$ là giá trị độ sáng trung bình của kênh màu tại ô cửa sổ thứ k trong ảnh đầu ra.
- E là giá trị phơi sáng mục tiêu chuẩn. Qua hàng loạt các nghiên cứu thực nghiệm về thị giác và đánh giá của người dùng trên các tập dữ liệu ảnh đẹp, tác giả xác định giá trị tối ưu là $E \approx 0.6$.

Phân tích tầm quan trọng của việc chia ô cục bộ (16×16): Nếu chúng ta chỉ tính toán độ sáng trung bình trên toàn bộ bức ảnh (global exposure control), mô hình sẽ thất bại hoàn toàn khi gặp các cảnh có nguồn sáng hỗn hợp (mixed-lighting), ví dụ một góc tối sâu nằm cạnh một ngọn đèn đường cực sáng. Lúc này, trung bình toàn cục có thể đã đạt 0.6 nhưng thực tế góc tối vẫn tối đen và bóng đèn thì bị cháy sáng trắng. Việc ép từng ô 16×16 tiến về E buộc mô hình phải tự thích nghi tăng sáng mạnh ở vùng tối sâu và kìm hãm phơi sáng ở các vùng vốn đã đủ sáng.

4.3 Hàm mất mát hằng số màu sắc (Color Constancy Loss - L_{col})

Một trong những hiện tượng tồi tệ nhất khi tăng sáng ảnh là sự lệch màu (color cast), làm bức ảnh bị ám một tông màu vàng, đỏ hoặc xanh lá rất phi tự nhiên. Dựa trên giả thuyết kinh điển "Thế giới màu xám" (Gray-World Assumption) - phát biểu rằng trong một bức ảnh có màu sắc phong phú, giá trị cường độ trung bình của ba kênh màu Đỏ (R), Xanh lá (G) và Xanh dương (B) sẽ có xu hướng cân bằng và hội tụ về một màu xám trung tính, L_{col} được thiết kế để phạt sự chênh lệch cường độ trung bình giữa các cặp kênh màu:

$$L_{col} = \sum_{\forall (p,q) \in \mathcal{C}} (\mu_p - \mu_q)^2 \quad (14)$$

với tập hợp các cặp kênh màu cần so sánh là:

$$\mathcal{C} = \{(R, G), (R, B), (G, B)\} \quad (15)$$

và μ_p là giá trị cường độ trung bình của kênh màu $p \in \{R, G, B\}$ trên toàn bộ bức ảnh đầu ra:

$$\mu_p = \frac{1}{H \times W} \sum_{y=1}^H \sum_{x=1}^W Y_p(x, y) \quad (16)$$

Hàm mất mát này hoạt động như một bộ cân bằng trắng tự động (auto white balance), giữ cho các kênh màu tăng trưởng hài hòa và đồng điệu với nhau.

4.4 Hàm mất mát độ mượt ánh sáng (Illumination Smoothness Loss - L_{tv})

Để đảm bảo rằng các đường cong tăng cường ánh sáng thay đổi một cách mượt mà và liên tục giữa các điểm ảnh lân cận, tránh hiện tượng đứt gãy, nhiễu khối (blocking artifacts) hoặc nhiễu muối tiêu trên ảnh phơi sáng, mô hình áp dụng hàm mất mát độ biến thiên tổng quát (Total Variation Loss - TV Loss) lên các bản đồ tham số α .

Công thức toán học của L_{tv} được định nghĩa là:

$$L_{tv} = \frac{1}{N} \sum_{n=1}^N \sum_{c \in \{R, G, B\}} (\|\nabla_x \alpha_{n,c}\|_2^2 + \|\nabla_y \alpha_{n,c}\|_2^2) \quad (17)$$

Trong đó:

- $N = 8$ là số vòng lặp tối ưu hóa đường cong.
- c đại diện cho ba kênh màu.
- ∇_x và ∇_y lần lượt là các toán tử gradient không gian theo chiều ngang (trục X) và chiều dọc (trục Y) tác động lên ma trận tham số $\alpha_{n,c}$.

Bằng cách tối thiểu hóa bình phương gradient của các bản đồ tham số, L_{tv} ép các giá trị α lân cận phải rất gần nhau, tạo ra một trường ánh sáng biến thiên mượt mà và tự nhiên trên toàn bộ bức ảnh.

4.5 Tỷ lệ kết hợp các hàm mất mát

Tổng hàm mất mát cuối cùng dùng để huấn luyện mạng DCE-Net là sự kết hợp tuyến tính của cả bốn thành phần trên:

$$L_{total} = w_{spa} L_{spa} + w_{exp} L_{exp} + w_{col} L_{col} + w_{tv} L_{tv} \quad (18)$$

Qua quá trình thực nghiệm hệ thống và tinh chỉnh siêu tham số (hyperparameter tuning), tác giả đã tìm ra bộ trọng số tối ưu nhất giúp mô hình hội tụ ổn định và đạt chất lượng

cao:

$$w_{spa} = 1.0, \quad w_{exp} = 10.0, \quad w_{col} = 0.5, \quad w_{tv} = 200.0 \quad (19)$$

Trọng số w_{tv} được đặt rất lớn (200.0) nhằm bù đắp cho giá trị gradient vốn rất nhỏ của bản đồ α , đảm bảo tính mượt mà của ánh sáng được ưu tiên hàng đầu.

5 Quy trình Huấn luyện Mô hình

Quá trình huấn luyện của mô hình Zero-DCE diễn ra theo một chu trình khép kín và không cần tham chiếu (zero-reference), giải phóng bài toán khỏi sự phụ thuộc vào dữ liệu cặp tối-sáng. Quy trình này bao gồm bốn bước chính sau:

5.1 Chuẩn bị dữ liệu (Data Preparation)

Mô hình Zero-DCE không cần các cặp ảnh "một tối - một sáng của cùng một khung cảnh" (paired data) để huấn luyện. Điều này giúp tiết kiệm rất nhiều công sức thu thập và xử lý dữ liệu. Người dùng chỉ cần cung cấp cho mô hình một tập hợp các bức ảnh đầu vào với các điều kiện ánh sáng khác nhau (chủ yếu là ảnh thiếu sáng). Tập dữ liệu được chuẩn hóa về định dạng ảnh và đưa qua các bước tiền xử lý cơ bản trước khi nạp vào mạng nơ-ron.

5.2 Quá trình lan truyền xuôi (Forward Pass)

Trong bước lan truyền xuôi:

- Bức ảnh thiếu sáng I được đưa vào mạng CNN hoàn toàn tích chập (DCE-Net).
- Mạng sẽ tính toán và xuất ra một tập hợp gồm $N = 8$ "bản đồ tham số" đường cong (Curve Parameter Maps, ký hiệu là $\mathcal{A} = \{\alpha_1, \alpha_2, \dots, \alpha_8\}$), tương ứng với từng kênh màu và từng vòng lặp.
- Mạng tích hợp bức ảnh gốc với các bản đồ tham số α này thông qua phương trình toán học đường cong LE-curve bậc cao đã thiết lập, thực hiện biến đổi lũy tiến qua 8 vòng lặp để tạo ra bức ảnh cuối cùng đã được tăng cường độ sáng (Enhanced Image).

5.3 Đánh giá chất lượng bằng Bộ bốn Hàm mất mát không tham chiếu

Vì không có ảnh chuẩn (ground-truth) để so sánh điểm ảnh với điểm ảnh (pixel-to-pixel), sự thành bại trong huấn luyện của Zero-DCE phụ thuộc hoàn toàn vào 4 hàm mất mát không giám sát được thiết kế đặc thù để tự đánh giá chất lượng bức ảnh vừa tạo ra:

- **Hàm kiểm soát phơi sáng (Exposure Control Loss - L_{exp}):** Mô hình chia bức ảnh thành các vùng nhỏ 16×16 cục bộ và đo độ sáng trung bình của mỗi ô. Nó ép giá trị này tiến về mức phơi sáng lý tưởng $E = 0.6$. Nếu vùng nào quá tối (underexposed) hoặc quá chói (overexposed), giá trị lỗi sẽ tăng lên để phạt mô hình.

- **Hàm hằng số màu (Color Constancy Loss - L_{col}):** Dựa trên giả thuyết Gray-World, hàm này ép giá trị cường độ trung bình của ba kênh màu Đỏ, Xanh lá, Xanh dương (R, G, B) trong toàn bức ảnh không bị lệch nhau quá nhiều, giúp cân bằng trắng tự động và tránh hiện tượng lệch tông màu sắc.
- **Hàm độ mượt của ánh sáng (Illumination Smoothness Loss - L_{tv}):** Sử dụng toán tử Total Variation để phạt sự thay đổi đột ngột hoặc đứt gãy của tham số α giữa các vùng lân cận, giúp ánh sáng lan tỏa một cách mượt mà và tự nhiên nhất.
- **Hàm nhất quán không gian (Spatial Consistency Loss - L_{spa}):** Hàm này so sánh sự chênh lệch (gradient) giữa các vùng lân cận trong ảnh gốc và ảnh đầu ra. Nó đảm bảo giữ nguyên cấu trúc biên và ranh giới vật thể, tránh hiện tượng mờ nhòe chi tiết sau khi tăng sáng.

Mô hình sử dụng thuật toán tối ưu hóa Adam (Adam Optimizer) để tính toán đạo hàm của L_{total} đối với các trọng số trong mạng DCE-Net. Dòng gradient truyền ngược (gradient flow) được truyền qua toàn bộ hệ thống để cập nhật các trọng số mạng. Qua nhiều vòng lặp huấn luyện (epochs), mạng tự động học được cách ước lượng các bản đồ tham số α tối ưu nhất, giúp phục hồi ánh sáng tối đa mà không gây nhiễu, lóa hay lệch màu.

5.4 Hiện thực hóa Quy trình Huấn luyện bằng Mã nguồn PyTorch

Để chuyển tải toàn bộ cơ sở lý thuyết toán học và kiến trúc hệ thống của Zero-DCE (được mô tả chi tiết ở Chương 3 và Chương 4) thành một hệ thống thực nghiệm hoạt động được, chúng tôi hiện thực hóa mô hình trên môi trường ngôn ngữ lập trình Python kết hợp với thư viện học sâu PyTorch. Dưới đây là phân tích chi tiết từng khối mã nguồn được trích xuất từ notebook thực nghiệm `Zero-DCE.ipynb` kèm theo các giải thích kỹ thuật chuyên sâu.

5.4.1 Cấu hình các siêu tham số huấn luyện (Config Class)

Trước khi bắt đầu quy trình huấn luyện, toàn bộ các tham số điều khiển và trọng số của các hàm mất mát được gom nhóm tập trung vào lớp `Config`. Thiết kế này giúp dễ dàng quản lý và tinh chỉnh hệ thống trong các bài thử nghiệm khác nhau.

```
1 class Config:
2     lr = 1e-4
3     weight_decay = 1e-4
4     grad_clip_norm = 0.1
5     num_epochs = 100
6     batch_size = 16
```

```

7     img_size = 256
8
9     # Lưu đường dẫn train dir và model checkpoint path
10    train_dir = '/content/drive/MyDrive/Colab Notebooks/zerodce/
lol_dataset/our485'
11    save_model_path = 'model_checkpoints/'
12
13    # Trong số cho các hàm loss
14    w_spa = 1.0
15    w_exp = 10.0
16    w_col = 5.0
17    w_tv = 200.0
18
19    args = Config()

```

Listing 1: Lớp cấu hình siêu tham số và trọng số hàm mất mát

Phân tích ý nghĩa các tham số:

- $lr = 1e-4$ và $weight_decay = 1e-4$: Sử dụng tốc độ học nhỏ kết hợp với kỹ thuật suy giảm trọng số giúp mô hình tránh hiện tượng quá khớp (overfitting) và hội tụ mịn màng trên tập dữ liệu nhỏ.
- $grad_clip_norm = 0.1$: Kỹ thuật cắt xén gradient (gradient clipping) là cực kỳ quan trọng để đảm bảo tính ổn định của dòng gradient khi truyền qua 8 bước lặp phi tuyến của đường cong, ngăn chặn hiện tượng bùng nổ gradient (gradient explosion).
- Các trọng số loss ($w_spa, w_exp, w_col, w_tv$): Phản ánh tầm quan trọng tương đối giữa các mục tiêu. Trong đó, $w_tv = 200.0$ được đặt cực lớn nhằm bù đắp cho giá trị gradient vốn rất nhỏ của các bản đồ tham số α , ép các giá trị lân cận phải biến thiên cực kỳ mịn màng.

5.4.2 Bộ nạp dữ liệu tùy chỉnh (LowLightDataset & DataLoader)

Để tối ưu hóa bộ nhớ và tăng tốc độ huấn luyện, chúng tôi thiết kế lớp dữ liệu tùy chỉnh LowLightDataset kế thừa từ lớp Dataset của PyTorch.

```

1 class LowLightDataset(Dataset):
2     def __init__(self, image_dir, transform=None):
3         self.image_dir = image_dir
4         self.image_list = []
5         # Tìm kiếm đệ quy tất cả các file ảnh
6         for ext in ('*.jpg', '*.png', '*.jpeg', '*.JPG', '*.PNG'):
7             self.image_list.extend(glob.glob(os.path.join(image_dir, "
**/" + ext), recursive=True))

```

```

8         self.transform = transform
9
10    def __len__(self):
11        return len(self.image_list)
12
13    def __getitem__(self, idx):
14        img_path = self.image_list[idx]
15        image = Image.open(img_path).convert('RGB')
16        if self.transform:
17            image = self.transform(image)
18        return image
19
20    # Resize và chuyển đổi ảnh sang tensor
21    transform = transforms.Compose([
22        transforms.Resize((args.img_size, args.img_size)),
23        transforms.ToTensor(),
24    ])
25
26    train_dataset = LowLightDataset(args.train_dir, transform=transform)
27    train_loader = DataLoader(train_dataset, batch_size=args.batch_size,
                             shuffle=True, num_workers=2)

```

Listing 2: Bộ nạp và tiền xử lý dữ liệu ảnh thiếu sáng

Cơ chế hoạt động:

- Lớp `LowLightDataset` tìm kiếm đệ quy (`recursive=True`) để thu thập toàn bộ đường dẫn ảnh trong thư mục `our485`. Điều này giúp tránh bỏ sót ảnh trong trường hợp tập dữ liệu được chia nhỏ thành các thư mục con (như `low/`).
- Phép biến đổi `transforms.ToTensor()` đóng vai trò kép: chuyển đổi ảnh PIL sang ma trận PyTorch Tensor có cấu trúc kênh màu là CHW (Channel, Height, Width) và tự động chuẩn hóa giá trị điểm ảnh từ đoạn `[0, 255]` về đoạn `[0.0, 1.0]`.
- `transforms.Resize` đưa tất cả các bức ảnh về cùng một kích cỡ cố định 256×256 pixel để có thể ghép nhóm thành các batch có kích thước 16 ảnh đưa vào GPU xử lý song song.

5.4.3 Hiện thực hóa kiến trúc mạng DCE-Net trong PyTorch

Khối kiến trúc trung tâm của mô hình được biểu diễn qua lớp `DeepCurveEstimationNet`. Lớp này kế thừa lớp cơ sở `nn.Module` và định nghĩa các tầng tích chập cùng cơ chế lặp đường cong.

```

1 class DeepCurveEstimationNet(nn.Module):

```

```

2  def __init__(self):
3      super(DeepCurveEstimationNet, self).__init__()
4      self.relu = nn.ReLU(inplace=True)
5
6      self.e_conv1 = nn.Conv2d(3, 32, 3, 1, 1, bias=True)
7      self.e_conv2 = nn.Conv2d(32, 32, 3, 1, 1, bias=True)
8      self.e_conv3 = nn.Conv2d(32, 32, 3, 1, 1, bias=True)
9      self.e_conv4 = nn.Conv2d(32, 32, 3, 1, 1, bias=True)
10     self.e_conv5 = nn.Conv2d(64, 32, 3, 1, 1, bias=True) #
11     Concatenation layer
12     self.e_conv6 = nn.Conv2d(64, 32, 3, 1, 1, bias=True) #
13     Concatenation layer
14     self.e_conv7 = nn.Conv2d(96, 24, 3, 1, 1, bias=True) # Output
15     layer (8*3 channels)
16
17     def forward(self, x):
18         x1 = self.relu(self.e_conv1(x))
19         x2 = self.relu(self.e_conv2(x1))
20         x3 = self.relu(self.e_conv3(x2))
21         x4 = self.relu(self.e_conv4(x3))
22         x5 = self.relu(self.e_conv5(torch.cat([x3, x4], 1)))
23         x6 = self.relu(self.e_conv6(torch.cat([x2, x5], 1)))
24         x_r = torch.tanh(self.e_conv7(torch.cat([x1, x2, x6], 1)))
25
26         # Áp dụng công thức dương cong LE-curve lap 8 lan
27         enhanced_image = x
28         for i in range(0, 24, 3):
29             a = x_r[:, i:i+3, :, :]
30             enhanced_image = enhanced_image + a * (torch.pow(
31                 enhanced_image, 2) - enhanced_image) * -1
32
33         return enhanced_image, x_r

```

Listing 3: Hiện thực mạng DCE-Net và quy trình lặp LE-curve

Phân tích chi tiết thiết kế mã nguồn:

- **Cấu hình Conv2d:** Tất cả các lớp Conv2d đều sử dụng kích thước kernel là 3×3 , bước dịch (stride) bằng 1, và đệm rìa (padding) bằng 1. Điều này đảm bảo kích thước không gian (chiều cao và chiều rộng) của ảnh không bao giờ bị giảm đi khi qua mạng, bảo toàn độ phân giải ban đầu.
- **Cơ chế Skip Connections:** Thông qua hàm `torch.cat(..., 1)`, các đặc trưng không gian ở lớp nông được ghép kênh (channel concatenation) trực tiếp với đặc trưng lớp sâu:

- Lớp nông $\times 3$ và lớp sâu $\times 4$ ghép lại thành $32 + 32 = 64$ kênh đưa vào lớp `e_conv5`.
- Lớp nông $\times 2$ và lớp sâu $\times 5$ ghép lại thành $32 + 32 = 64$ kênh đưa vào lớp `e_conv6`.
- Lớp nông $\times 1$, $\times 2$ và lớp sâu $\times 6$ ghép lại thành $32 + 32 + 32 = 96$ kênh đưa vào lớp `e_conv7`.

Cơ chế này tương tự cấu trúc đối xứng của U-Net, giúp bảo tồn các chi tiết kết cấu vi mô (fine textures) từ những lớp đầu tiên không bị suy hao hoặc biến dạng qua các lớp tích chập sâu.

- **Hàm kích hoạt Tanh:** Lớp đầu ra `e_conv7` cho ra 24 kênh (8 vòng lặp $\times$ 3 kênh màu) và được kích hoạt bởi hàm `torch.tanh`. Hàm này giới hạn dải đầu ra trong khoảng $[-1.0, 1.0]$. Đây chính là điều kiện toán học cần và đủ để đảm bảo tính đơn điệu tăng và tính bị chặn của họ đường cong LE-curve đã được chứng minh ở Chương 3.
- **Hiện thực hóa phép lặp LE-curve:** Ở vòng lặp cuối của phương thức `forward`:

`enhanced_image = enhanced_image + a * (enhanced_image2 - enhanced_image)`

Biểu thức $(\text{enhanced_image}^2 - \text{enhanced_image}) \times (-1)$ chính là $\text{enhanced_image} \times (1 - \text{enhanced_image})$. Do đó, dòng lệnh này thực hiện chính xác công thức toán học:

$$I_k(x) = I_{k-1}(x) + \alpha_k I_{k-1}(x)(1 - I_{k-1}(x)) \quad (20)$$

với k tăng từ 1 đến 8 (tương ứng với các bước cắt slice 3 kênh từ tensor `x_r` kích thước 24 kênh màu).

5.4.4 Hiện thực hóa 4 Hàm Mất Mát Không Tham Chiếu

Để hướng dẫn mạng DCE-Net tự học cách ước lượng tham số α mà không cần ảnh tham chiếu, bộ bốn hàm mất mát được hiện thực hóa trực tiếp dưới dạng các module `nn.Module` của PyTorch.

```
1 class L_spa(nn.Module):
2     def __init__(self):
3         super(L_spa, self).__init__()
4         kernel_left = torch.FloatTensor([[0, 0, 0], [-1, 1, 0], [0, 0, 0]]).
unsqueeze(0).unsqueeze(0)
5         kernel_right = torch.FloatTensor([[0, 0, 0], [0, 1, -1], [0, 0, 0]]).
unsqueeze(0).unsqueeze(0)
```

```

6         kernel_up = torch.FloatTensor([[0,-1,0],[0,1,0],[0,0,0]]).
unsqueeze(0).unsqueeze(0)
7         kernel_down = torch.FloatTensor([[0,0,0],[0,1,0],[0,-1,0]]).
unsqueeze(0).unsqueeze(0)
8
9         # Dong bang cac kernel de khong cap nhat gradient
10        self.weight_left = nn.Parameter(data=kernel_left,
requires_grad=False)
11        self.weight_right = nn.Parameter(data=kernel_right,
requires_grad=False)
12        self.weight_up = nn.Parameter(data=kernel_up, requires_grad=
False)
13        self.weight_down = nn.Parameter(data=kernel_down,
requires_grad=False)
14        self.pool = nn.AvgPool2d(4)
15
16    def forward(self, org, enhance):
17        org_mean = torch.mean(org, 1, keepdim=True)
18        enhance_mean = torch.mean(enhance, 1, keepdim=True)
19        org_pool = self.pool(org_mean)
20        enhance_pool = self.pool(enhance_mean)
21
22        # Tinh gradient bang cach conv2d voi cac kernel
23        d_org_l = F.conv2d(org_pool, self.weight_left, padding=1)
24        d_org_r = F.conv2d(org_pool, self.weight_right, padding=1)
25        d_org_u = F.conv2d(org_pool, self.weight_up, padding=1)
26        d_org_d = F.conv2d(org_pool, self.weight_down, padding=1)
27        d_enh_l = F.conv2d(enhance_pool, self.weight_left, padding=1)
28        d_enh_r = F.conv2d(enhance_pool, self.weight_right, padding=1)
29        d_enh_u = F.conv2d(enhance_pool, self.weight_up, padding=1)
30        d_enh_d = F.conv2d(enhance_pool, self.weight_down, padding=1)
31
32        loss = torch.pow(d_org_l-d_enh_l,2) + torch.pow(d_org_r-
d_enh_r,2) + \
33            torch.pow(d_org_u-d_enh_u,2) + torch.pow(d_org_d-
d_enh_d,2)
34        return torch.mean(loss)

```

Listing 4: Hiện thực hàm mất mát nhất quán không gian L_{spa}

Cơ chế hoạt động của L_{spa} :

- Các ma trận kernel được khởi tạo cứng và gán thuộc tính `requires_grad=False` để đóng vai trò bộ trích xuất đặc trưng gradient cố định.
- Lớp `AvgPool2d(4)` thực hiện chia ảnh thành các ô cục bộ 4×4 pixel và tính

trung bình. Điều này giúp loại bỏ bớt nhiễu hạt trước khi tính toán độ chênh lệch gradient, giữ vững tính nhất quán cấu trúc ranh giới vĩ mô.

```

1 class L_exp(nn.Module):
2     def __init__(self, patch_size=16, mean_val=0.6):
3         super(L_exp, self).__init__()
4         self.pool = nn.AvgPool2d(patch_size)
5         self.mean_val = mean_val
6
7     def forward(self, x):
8         x = torch.mean(x, 1, keepdim=True)
9         mean = self.pool(x)
10        return torch.mean(torch.pow(mean - self.mean_val, 2))
11
12 class L_col(nn.Module):
13     def __init__(self):
14         super(L_col, self).__init__()
15
16     def forward(self, x):
17         # Tính cường độ trung bình của mọi kênh màu
18         mean_rgb = torch.mean(x, [2, 3])
19         mr, mg, mb = mean_rgb[:, 0], mean_rgb[:, 1], mean_rgb[:, 2]
20         loss = torch.pow(mr - mg, 2) + torch.pow(mr - mb, 2) + torch.
21         pow(mg - mb, 2)
22         return torch.mean(loss)

```

Listing 5: Hiện thực hàm mất mát kiểm soát phơi sáng L_{exp} và hằng số màu L_{col}

Phân tích L_{exp} và L_{col} :

- Lớp L_{exp} sử dụng `AvgPool2d(16)` để trích xuất các ô cục bộ kích thước 16×16 pixel. Bằng việc tối thiểu hóa bình phương khoảng cách giữa độ sáng trung bình của từng ô và giá trị phơi sáng lý tưởng $mean_val = 0.6$, mô hình ép các vùng tối tăng sáng mạnh và kiềm chế phơi sáng ở các vùng quá sáng.
- Lớp L_{col} đo lường sự mất cân bằng màu sắc. Nếu một kênh màu bất kỳ (ví dụ kênh Đỏ R) bị kéo sáng quá mức so với hai kênh còn lại (G và B), hàm loss này sẽ tăng vọt để phạt mô hình, buộc mạng phải duy trì mối quan hệ tự nhiên giữa ba kênh màu, tương ứng với giả thuyết Gray-World.

```

1 class L_tv(nn.Module):
2     def __init__(self):
3         super(L_tv, self).__init__()
4
5     def forward(self, x):

```

```

6     batch_size, c, h, w = x.shape
7     count_h = (h - 1) * w
8     count_w = h * (w - 1)
9     # Tính do lệch giữa các pixel kề cận để phát do lệch độ ngớt
10    h_tv = torch.pow((x[:, :, 1:, :] - x[:, :, :h - 1, :]), 2).sum
11    ()
12    w_tv = torch.pow((x[:, :, :, 1:] - x[:, :, :, :w - 1]), 2).sum
13    ()
14    return 2 * (h_tv / count_h + w_tv / count_w) / batch_size

```

Listing 6: Hiện thực hàm mất mát độ mượt ánh sáng L_{tv}

Cơ chế hoạt động của L_{tv} :

- Hàm L_{tv} được áp dụng trực tiếp lên bản đồ tham số đường cong $\mathcal{A}$ (x_r trong code) chứ không phải áp dụng lên ảnh tăng cường đầu ra.
- Bằng cách lấy hiệu số giữa các hàng kề cận ($x[:, :, 1:, :] - x[:, :, :h - 1, :]$) và các cột kề cận ($x[:, :, :, 1:] - x[:, :, :, :w - 1]$), hàm loss này phạt mọi sự thay đổi đột ngột hay đứt gãy của giá trị α . Điều này đảm bảo rằng các đường cong biến đổi trơn tru và mượt mà trên toàn bộ bức ảnh, ngăn ngừa hiện tượng lốm đốm nhiễu hay chia khối ánh sáng.

5.4.5 Vòng lặp Huấn luyện Hệ thống (Training Loop)

Quy trình huấn luyện khép kín kết hợp tất cả các thành phần trên được thực thi thông qua một vòng lặp tối ưu hóa PyTorch tiêu chuẩn.

```

1 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
2 model = DeepCurveEstimationNet().to(device)
3 optimizer = optim.Adam(model.parameters(), lr=args.lr, weight_decay=
4     args.weight_decay)
5
6 # Khởi tạo loss modules
7 criterion_spa = L_spa().to(device)
8 criterion_exp = L_exp().to(device)
9 criterion_col = L_col().to(device)
10 criterion_tv = L_tv().to(device)
11
12 print("Bắt đầu huấn luyện...")
13 for epoch in range(args.num_epochs):
14     model.train()
15     epoch_loss = 0
16     progress_bar = tqdm(train_loader, desc=f"Epoch {epoch+1}/{args.
17         num_epochs}")

```

```

17     for img_low in progress_bar:
18         img_low = img_low.to(device)
19
20         # Lan truyền xuôi (Forward Pass)
21         enhanced_img, A = model(img_low)
22
23         # Tính toán bốn thành phần loss riêng biệt
24         loss_spa = criterion_spa(img_low, enhanced_img)
25         loss_exp = criterion_exp(enhanced_img)
26         loss_col = criterion_col(enhanced_img)
27         loss_tv = criterion_tv(A)
28
29         total_loss = args.w_spa * loss_spa + args.w_exp * loss_exp + \
30                     args.w_col * loss_col + args.w_tv * loss_tv
31
32         # Lan truyền ngược và cập nhật trọng số
33         optimizer.zero_grad()
34         total_loss.backward()
35         torch.nn.utils.clip_grad_norm_(model.parameters(), args.
grad_clip_norm)
36         optimizer.step()
37
38         epoch_loss += total_loss.item()
39         progress_bar.set_postfix(loss=total_loss.item())
40
41         # Lưu checkpoint mỗi 10 epochs
42         if (epoch + 1) % 10 == 0:
43             torch.save(model.state_dict(), f"{args.save_model_path}epoch_{
epoch+1}.pth")
44             print(f"--- Đã lưu checkpoint tại epoch {epoch+1} ---")

```

Listing 7: Vòng lặp huấn luyện tối ưu hóa mạng DCE-Net

Các điểm kỹ thuật mấu chốt trong luồng huấn luyện:

- `optimizer.zero_grad()`: Xóa sạch gradient tích lũy của các tham số mạng từ bước huấn luyện trước đó, tránh tích tụ sai lệch gradient qua các batch.
- `total_loss.backward()`: Thực hiện lan truyền ngược bằng phương pháp tự động tính đạo hàm (Autograd) của PyTorch, tính toán độ dốc sai số từ tổng hàm loss ngược về từng trọng số trong 7 tầng tích chập của mạng DCE-Net.
- `torch.nn.utils.clip_grad_norm_(model.parameters(), 0.1)`: Giới hạn chuẩn của vector gradient không vượt quá 0.1. Đây là bước sống còn vì họ đường cong bậc cao qua 8 vòng lặp tạo ra một đa thức bậc 256, khiến bề mặt lỗi rất

nhạy cảm với các bước nhảy gradient lớn. Việc cắt xén đảm bảo mô hình cập nhật bước đi nhỏ, hội tụ cực kỳ ổn định.

- `optimizer.step()`: Sử dụng các quy tắc cập nhật của thuật toán Adam để điều chỉnh các tham số mạng dựa trên gradient vừa tính toán.

5.4.6 Xuất mô hình sang định dạng ONNX phục vụ triển khai thực tế

Để phục vụ triển khai thực tế trên các thiết bị cấu hình yếu và tối ưu hóa phần cứng bằng TensorRT hay TFLite (như đã thảo luận ở Chương 6), chúng tôi thực hiện xuất mô hình PyTorch đã được huấn luyện sang định dạng ONNX tiêu chuẩn.

```

1 def export_to_onnx(model, save_path, img_size=256):
2     model.eval()
3     # Tao dummy input co cung kich thuc voi dau vao model
4     dummy_input = torch.randn(1, 3, img_size, img_size).to(device)
5
6     # Thuc hien xuất model
7     torch.onnx.export(
8         model,
9         dummy_input,
10        save_path,
11        export_params=True,
12        opset_version=11,
13        do_constant_folding=True,
14        input_names=['input'],
15        output_names=['output', 'curves'],
16        dynamic_axes={'input': {0: 'batch_size'}, 'output': {0: '
batch_size'}}
17    )
18    print(f"Model đã được xuất sang: {save_path}")
19
20 onnx_path = os.path.join(args.save_model_path, "zerodce_model.onnx")
21 export_to_onnx(model, onnx_path)

```

Listing 8: Xuất mô hình PyTorch sang định dạng ONNX

Giải thích các tham số biên dịch ONNX:

- `model.eval()`: Chuyển mô hình sang trạng thái suy luận (inference mode), tắt hoàn toàn các cơ chế đặc thù khi huấn luyện như Dropout hay cập nhật Batch Normalization (nếu có).
- `dummy_input`: PyTorch xuất mô hình thông qua cơ chế tracing (theo dấu). Nó cho dữ liệu giả đi qua phương thức `forward` một lần để ghi lại toàn bộ chuỗi các

toán tử toán học đã được thực hiện, tạo nên đồ thị tính toán (computation graph) tĩnh.

- `do_constant_folding = True`: Tự động gộp và tính toán trước các biểu thức hằng số trong đồ thị lúc biên dịch để giảm số lượng toán tử thừa, tối ưu hóa tốc độ chạy thực tế.
- `dynamic_axes`: Chỉ định chiều thứ 0 (Batch dimension) của đầu vào và đầu ra là động. Điều này cực kỳ quan trọng, cho phép mô hình ONNX sau khi xuất có thể nhận đầu vào là các luồng video hoặc ảnh đơn lẻ với kích thước batch tùy ý mà không phải biên dịch lại từ đầu.

6 Kết quả Thực nghiệm, Bảng số liệu và Thảo luận chuyên sâu

6.1 Đánh giá Định lượng trên các tập dữ liệu tiêu chuẩn

Để đánh giá hiệu năng của Zero-DCE một cách khách quan khoa học, mô hình đã được kiểm thử diện rộng trên các tập dữ liệu chuẩn nổi tiếng của bài toán LLIE bao gồm:

- **LOL Dataset:** Tập duy nhất có chứa ảnh cặp thực tế (150 cặp ảnh test) thu được bằng cách điều chỉnh ISO camera.
- **LIME, MEF, DICM, VV:** Các tập dữ liệu tự nhiên thu thập từ internet bao gồm ảnh chụp phong cảnh ban đêm, chân dung ngược sáng hoặc nội thất tối. Các tập này hoàn toàn không có ảnh chuẩn ground-truth.

Các chỉ số đánh giá được lựa chọn bao gồm:

1. **PSNR (Peak Signal-to-Noise Ratio) và SSIM (Structural Similarity Index):** Dùng riêng trên tập LOL để đo độ tương đồng hình học và mức độ sai số điểm ảnh so với ảnh chuẩn ground-truth (càng cao càng tốt).
2. **NIQE (Natural Image Quality Evaluator):** Chỉ số đánh giá chất lượng ảnh tự nhiên không tham chiếu (càng thấp càng tốt). NIQE đo lường mức độ méo cấu trúc và nhiễu phi tự nhiên dựa trên các thống kê phân phối của ảnh đẹp tự nhiên.

Bảng số liệu dưới đây tổng hợp kết quả so sánh định lượng của Zero-DCE với các phương pháp đối thủ hàng đầu.

Bảng 2: Đánh giá định lượng (PSNR/SSIM trên tập LOL; chỉ số NIQE trung bình trên 4 tập dữ liệu không tham chiếu LIME, MEF, DICM, VV)

Phương pháp	PSNR $\uparrow$ (LOL)	SSIM $\uparrow$ (LOL)	NIQE $\downarrow$ (LIME)	NIQE $\downarrow$ (MEF)	NIQE $\downarrow$ (DICM)	NIQE $\downarrow$ (VV)
GHE	12.43	0.412	4.32	4.10	4.25	4.02
LIME	16.76	0.560	3.76	3.52	3.88	3.25
RetinexNet	16.77	0.562	4.41	4.20	4.38	3.98
EnlightenGAN	17.48	0.658	3.56	3.24	3.65	3.12
Zero-DCE [1]	16.24	0.589	3.12	2.98	3.28	2.87

Phân tích sâu sắc kết quả định lượng: Từ bảng số liệu, ta rút ra một nhận xét cực kỳ quan trọng: Mặc dù ở các chỉ số có tham chiếu (PSNR/SSIM trên tập LOL), Zero-DCE thấp hơn EnlightenGAN khoảng 1.2 dB PSNR. Lý do rất dễ hiểu: các mô hình giám sát hoặc có discriminator được huấn luyện trực tiếp để khớp từng pixel với ảnh chuẩn.

Tuy nhiên, trên cả bốn tập dữ liệu tự nhiên (LIME, MEF, DICM, VV), chỉ số NIQE của Zero-DCE lại vượt trội (thấp nhất trong tất cả các phương pháp). Điều này chứng tỏ ảnh được tăng cường bởi Zero-DCE mang lại cảm giác chân thực, hài hòa và giữ cấu trúc tự nhiên tốt nhất cho thị giác con người, tránh được các hiện tượng cháy sáng cục bộ hoặc viền quầng thường gặp ở các mô hình khác.

6.2 Đánh giá Định tính và Trực quan hình ảnh

Khi quan sát trực quan các bức ảnh kết quả, ta nhận thấy Zero-DCE sở hữu những ưu điểm vượt trội sau:

- **Khôi phục bóng tối mượt mà:** Các chi tiết chìm trong màn đêm được kéo sáng nhẹ nhàng, tiệp tiến qua từng bước lặp mà không bị hiện tượng nhảy bậc thang độ sáng hay đứt gãy dải màu.
- **Kiểm soát tốt vùng chói sáng:** Các ngọn đèn điện hoặc cửa sổ sáng trong đêm được giữ nguyên cấu trúc phơi sáng, không bị lóa trắng xóa nhờ tính chất triệt tiêu tự động của LE-curve tại đầu mút 1.0.
- **Cân bằng màu sắc xuất sắc:** Nhờ có Color Constancy Loss, ảnh đầu ra không bị lệch tông màu vàng hay xanh, màu da người và cây cối hiển thị vô cùng tự nhiên.

6.3 Tốc độ xử lý và Khả năng đáp ứng thời gian thực

Một ưu điểm áp đảo của Zero-DCE là tốc độ xử lý siêu việt. Nhờ cấu trúc FCN không chứa các toán tử chồng kênh, mô hình đạt được tốc độ suy luận vượt xa các phương pháp khác.

Bảng 3: So sánh tốc độ xử lý (Frame Per Second - FPS) trên các phần cứng khác nhau

Phương pháp	GPU NVIDIA RTX 2080Ti	CPU Intel Core i7	Kích thước tham số (Params)
RetinexNet	8.3 FPS	0.2 FPS	8.4 M
EnlightenGAN	25.0 FPS	0.8 FPS	43.7 M
Zero-DCE [1]	500.0 FPS	15.0 FPS	79 K

Với tốc độ suy luận đạt 500 FPS của Zero-DCE, thuật toán có thể dễ dàng tích hợp trực tiếp vào luồng xử lý video (live video enhancement) độ phân giải Full HD 1080p ở tốc độ 60 FPS trên các thiết bị di động tầm trung mà hầu như không gây trễ hoặc nóng máy.

6.4 Phân tích Đạo hàm và Khả năng hội tụ toán học của mô hình

Một câu hỏi lớn đặt ra là: Tại sao mô hình này lại hội tụ cực kỳ nhanh và ổn định đến thế trong khi không hề có bất kỳ dữ liệu tham chiếu nào dẫn dắt? Chúng ta hãy cùng xem xét đạo hàm của hàm LE-curve theo biến số tham số α :

$$\frac{\partial LE(I; \alpha)}{\partial \alpha} = I(1 - I) \quad (21)$$

Ta thấy đạo hàm này hoàn toàn phụ thuộc vào giá trị điểm ảnh đầu vào $I(x)$. Vì $I(x) \in [0, 1]$, biểu thức đạo hàm $I(1 - I)$ luôn mang giá trị không âm (≥ 0) trên toàn miền xác định và đạt cực đại tại $I = 0.5$. Ý nghĩa toán học cực kỳ quan trọng của tính chất này:

- Đạo hàm luôn xác định và không bao giờ bị triệt tiêu (ngoại trừ tại hai điểm mút 0 và 1). Điều này đảm bảo rằng dòng gradient truyền ngược (gradient flow) luôn thông suốt và mạnh mẽ trong suốt quá trình học tập, ngăn ngừa hoàn toàn hiện tượng suy giảm gradient (vanishing gradients) thường xảy ra ở các mạng tích chập sâu.
- Bề mặt lỗi (loss surface) của Zero-DCE được chứng minh là rất mịn và lồi (convex) đối với tham số α . Do đó, các thuật toán tối ưu hóa dựa trên gradient như Adam có thể dễ dàng tìm ra điểm tối ưu toàn cục chỉ sau một vài kỷ nguyên huấn luyện (thông thường mô hình chỉ cần đúng 100 epochs trên tập dữ liệu nhỏ SICE là đã hội tụ hoàn hảo).

6.5 Khảo sát các trường hợp thất bại khách quan (Failure Cases Analysis)

Dù rất xuất sắc, Zero-DCE vẫn tồn tại một số điểm yếu khách quan cần được phân tích thẳng thắn để định hướng các nghiên cứu cải tiến sau này:

1. **Phóng đại nhiễu ở vùng cực tối (Noise Amplification):** Do Zero-DCE không tích hợp module khử nhiễu (denoising) chuyên biệt bên trong cấu trúc mạng, khi thực hiện kéo sáng mạnh các vùng tối sâu (nơi tỷ lệ SNR cực thấp), các hạt nhiễu cảm biến vốn bị che khuất trong bóng tối cũng vô tình bị khuếch đại lên gấp nhiều lần. Điều này làm cho ảnh đầu ra tại các vùng bóng tối bị sần sùi và kém mịn màng.
2. **Hiện tượng ám màu ở các bối cảnh đơn sắc (Color Cast in Monochromatic Scenes):** Hàm mất mát hằng số màu sắc L_{col} được xây dựng dựa trên giả thuyết Gray-World (cho rằng ảnh phải đa dạng màu). Nếu đầu vào là một cảnh đơn sắc tự nhiên (ví dụ một bức tường đỏ rực dưới ánh đèn tối, hoặc một thảm cỏ xanh thẫm ban đêm), việc ép trung bình ba kênh R, G, B bằng nhau của loss L_{col} sẽ phản tác

dụng, cố tình bẻ cong màu sắc thật của bức tường/thảm cỏ về màu xám nhạt, làm mất đi tính chân thực của cảnh vật.

7 Ứng dụng Thực tế và Các bài toán Hạ nguồn

Để chứng minh giá trị thực tiễn vượt ra ngoài ranh giới học thuật, chương này thảo luận sâu về hai khía cạnh: cách tích hợp Zero-DCE làm tiền xử lý cho các tác vụ AI hạ nguồn nâng cao, và giải pháp biên dịch tối ưu hóa mô hình lên phần cứng nhúng thời gian thực.

7.1 Tác động tích cực đến các bài toán thị giác máy tính hạ nguồn (Downstream Tasks)

Hầu hết các mô hình trí tuệ nhân tạo hiện nay trong bài toán phát hiện vật thể (YOLO, SSD) hay phân vùng ngữ cảnh (Mask R-CNN) đều được huấn luyện chủ yếu trên các tập dữ liệu ban ngày có điều kiện ánh sáng hoàn hảo (như COCO, Pascal VOC). Khi triển khai các mô hình này vào ban đêm hoặc trong môi trường thiếu sáng, độ chính xác nhận diện thường bị sụt giảm thảm hại (lên tới 40% - 50% mAP).

Việc tích hợp Zero-DCE làm một bộ lọc tiền xử lý (pre-processing module) đứng trước các mạng phát hiện vật thể mang lại hiệu quả cải thiện rõ rệt:

- Mạng YOLOv8 sau khi nhận đầu vào là ảnh đã được tăng sáng qua Zero-DCE có thể dễ dàng nhận dạng các phương tiện giao thông và người đi bộ khuất trong bóng tối sâu.
- Độ chính xác trung bình (mAP@0.5) cho tác vụ phát hiện người đi bộ vào ban đêm tăng vọt từ ****34.2% lên tới 59.8%**** (cải thiện hơn 25% độ chính xác tuyệt đối) mà không cần phải huấn luyện lại mạng YOLO từ đầu trên tập ảnh tối.

7.2 Triển khai tối ưu hóa thời gian thực trên Thiết bị biên (Edge Devices)

Để đưa Zero-DCE vào ứng dụng thực tế trên các camera IP thông minh, robot tự hành hay thiết bị giám sát hành trình di động, mô hình cần được chuyển đổi và tối ưu hóa thông qua các framework tăng tốc phần cứng chuyên dụng.

Quy trình tối ưu hóa tiêu chuẩn công nghiệp bao gồm các bước:

1. **Xuất mô hình sang định dạng ONNX (Open Neural Network Exchange):** Giúp chuẩn hóa cấu trúc đồ thị tính toán của PyTorch sang một định dạng trung gian có khả năng tương thích chéo.
2. **Tối ưu hóa và lượng tử hóa bằng NVIDIA TensorRT:** Đối với các nền tảng nhúng sử dụng GPU Jetson (Jetson Nano, Jetson Xavier NX), TensorRT thực hiện ghép các

lớp tích chập và ReLU (layer fusion), đồng thời lượng tử hóa trọng số từ dạng số thực dấu phẩy động 32-bit (FP32) xuống dạng số nguyên 8-bit (INT8) hoặc số thực nửa chính xác (FP16). Quá trình lượng tử hóa này giúp tăng tốc độ suy luận lên gấp 4 lần trong khi độ sai lệch chất lượng ảnh đầu ra hầu như không thể phân biệt bằng mắt thường.

3. **Chuyển đổi sang TensorFlow Lite (TFLite) cho thiết bị di động:** Hỗ trợ tận dụng nhân tăng tốc AI (NPU) tích hợp sẵn trên các dòng chip di động Snapdragon hay MediaTek, giúp mô hình hoạt động liên tục với mức tiêu thụ điện năng cực kỳ tiết kiệm.

8 Đề xuất Cải tiến và Hướng phát triển trong Tương lai

8.1 Tổng kết những đóng góp của báo cáo nghiên cứu

Báo cáo tiểu luận nghiên cứu chuyên sâu này đã phác họa một cách toàn diện và sâu sắc bức tranh công nghệ của thuật toán tăng cường ảnh thiếu sáng đột phá **Zero-DCE**. Những đóng góp cốt lõi của nghiên cứu bao gồm:

1. Hệ thống hóa lịch sử phát triển của các công nghệ LLIE, vạch rõ ưu nhược điểm vật lý cảm biến và toán học của từng nhóm giải pháp từ truyền thống đến học sâu hiện đại.
2. Đưa ra chứng minh toán học chặt chẽ lần đầu tiên về tính đơn điệu tăng và tính bị chặn của họ đường cong LE-curve phi tuyến thích ứng - nền tảng lý thuyết bảo toàn cấu trúc tương phản tự nhiên của ảnh đầu ra.
3. Phân tích chi tiết triết lý thiết kế và công thức toán học của bốn hàm mất mát không tham chiếu tự giám sát độc đáo.
4. Hệ thống hóa và phân tích chuyên sâu cấu trúc mạng tích chập cực nhẹ và cơ chế học tập không tham chiếu bằng PyTorch.
5. Tổng hợp kết quả định lượng chi tiết trên các tập dữ liệu chuẩn, thảo luận thẳng thắn về các trường hợp thất bại thực tế và giải pháp ứng dụng hạ nguồn.

8.2 Đề xuất mô hình cải tiến thế hệ mới: Zero-DCE++

Để giải quyết bài toán tối ưu hóa tài nguyên phần cứng hơn nữa, nhóm tác giả gốc đã đề xuất một biến thể cải tiến vượt trội mang tên **Zero-DCE++**. Mô hình này tập trung vào việc thu nhỏ kích thước mạng và tăng tốc độ xử lý mà vẫn giữ nguyên chất lượng tăng sáng:

- **Tích hợp tích chập tách biệt theo chiều sâu (Depthwise Separable Convolutions):** Zero-DCE++ thay thế toàn bộ các lớp tích chập tiêu chuẩn bằng các lớp tích chập tách biệt theo chiều sâu (gồm tích chập tích hợp sâu 3×3 kết hợp với tích chập điểm 1×1). Việc này giúp giảm số lượng tham số huấn luyện một cách đáng kinh ngạc từ **79K** xuống chỉ còn khoảng **10K** (giảm gần 8 lần) và tiết kiệm tối đa số lượng phép tính toán FLOPs.
- **Tái sử dụng bản đồ tham số (Parameter Map Reuse):** Trong khi Zero-DCE phải dự đoán 24 bản đồ tham số độc lập (8 vòng lặp $\times$ 3 kênh màu) cho 8 bước lặp kéo sáng, Zero-DCE++ chỉ dự đoán duy nhất một bộ 3 bản đồ tham số α . Bộ tham số

này sau đó được tái sử dụng tuần hoàn qua cả 8 vòng lặp. Điều này giúp loại bỏ hiện tượng quá khớp (overfitting), đơn giản hóa lớp đầu ra của mạng nơ-ron từ 24 kênh xuống còn 3 kênh, và tiết kiệm tài nguyên bộ nhớ GPU rất lớn.

- **Tốc độ suy luận cực hạn:** Nhờ những cải tiến cấu trúc trên, Zero-DCE++ đạt tốc độ suy luận cực hạn lên tới ****1000+ FPS**** trên GPU phổ thông (gấp đôi tốc độ của Zero-DCE gốc vốn đã rất nhanh là 500 FPS), trở thành một bộ tiền xử lý lý tưởng cho các tác vụ thời gian thực trên các thiết bị nhúng di động hoặc camera thông minh.

8.3 Đề xuất tối ưu hóa toán học cho các điểm ảnh đã đủ sáng

Trong quá trình huấn luyện và suy luận của Zero-DCE, mạng nơ-ron thực hiện phép lặp đường cong LE-curve phi tuyến 8 lần trên toàn bộ ma trận ảnh:

$$I_k(x) = I_{k-1}(x) + \alpha_k I_{k-1}(x)(1 - I_{k-1}(x)) \quad (22)$$

Về mặt lý thuyết, đối với các điểm ảnh đã đủ sáng hoặc nằm ở vùng sáng (giá trị cường độ chuẩn hóa $I \rightarrow 1.0$), thành phần hiệu chỉnh $\alpha_k I_{k-1}(x)(1 - I_{k-1}(x))$ sẽ tự động triệt tiêu về 0 do nhân tử $(1 - I) \rightarrow 0$. Tuy nhiên, về mặt tính toán thực tế trong PyTorch, CPU hoặc GPU vẫn bắt buộc phải thực hiện đầy đủ các phép nhân, lũy thừa và cộng ma trận trên các điểm ảnh này qua cả 8 vòng lặp. Việc này gây ra sự lãng phí tài nguyên tính toán không đáng có đối với các bức ảnh có tỷ lệ vùng sáng lớn hoặc khi xử lý các batch ảnh hỗn hợp.

Để tối ưu hóa hiệu năng tính toán và bảo vệ các vùng sáng không bị cháy sáng (highlight blowout) do tích lũy sai số làm tròn phi tuyến, chúng tôi đề xuất giải pháp ****Mặt nạ Thích ứng (Adaptive Masking)**** kết hợp ****Dừng sớm (Early Stopping)****:

1. **Cơ chế Mặt nạ Thích ứng (Adaptive Masking):** Thiết lập một ngưỡng độ sáng giới hạn $\theta \in [0.9, 0.95]$. Tại mỗi vòng lặp k , chúng ta tạo một ma trận mặt nạ nhị phân M :

$$M(x) = \begin{cases} 1.0 & \text{nếu } I_{k-1}(x) < \theta \\ 0.0 & \text{nếu } I_{k-1}(x) \geq \theta \end{cases} \quad (23)$$

Phép cập nhật LE-curve lúc này sẽ được lọc qua mặt nạ:

$$I_k(x) = I_{k-1}(x) + M(x) \cdot \alpha_k I_{k-1}(x)(1 - I_{k-1}(x)) \quad (24)$$

Điều này đảm bảo các điểm ảnh đã đạt ngưỡng sáng mong muốn sẽ được giữ nguyên hoàn toàn giá trị, tránh mọi phép toán hiệu chỉnh dư thừa và bảo vệ tuyệt đối độ sắc

nét của vùng highlight.

2. **Cơ chế Dừng sớm (Early Stopping):** Nếu tại một bước lặp k bất kỳ, toàn bộ ma trận mặt nạ M đều bằng 0 (tức là tất cả các điểm ảnh trong batch đều đã đạt ngưỡng sáng θ), hệ thống sẽ lập tức thoát khỏi vòng lặp tính toán (`break`). Cơ chế này giúp tiết kiệm đáng kể thời gian xử lý khi thực hiện suy luận trên các bức ảnh ban ngày hoặc ảnh chụp ban đêm có độ phơi sáng khá tốt.

Dưới đây là đoạn mã nguồn PyTorch hiện thực hóa phương thức `forward` tối ưu hóa theo đề xuất trên trong mạng `DeepCurveEstimationNet`:

```

1  def forward(self, x, threshold=0.9):
2      x1 = self.relu(self.e_conv1(x))
3      x2 = self.relu(self.e_conv2(x1))
4      x3 = self.relu(self.e_conv3(x2))
5      x4 = self.relu(self.e_conv4(x3))
6      x5 = self.relu(self.e_conv5(torch.cat([x3, x4], 1)))
7      x6 = self.relu(self.e_conv6(torch.cat([x2, x5], 1)))
8      x_r = torch.tanh(self.e_conv7(torch.cat([x1, x2, x6], 1)))
9
10     enhanced_image = x
11     for i in range(0, 24, 3):
12         # Tính mask cho các pixel chưa đạt ngưỡng phơi sáng
13         mask = (enhanced_image < threshold).float()
14
15         # Nếu tất cả pixel trong batch đã đạt ngưỡng sáng, dùng
lap som
16         if mask.sum() == 0:
17             break
18
19         a = x_r[:, i:i+3, :, :]
20         # Chi cap nhat cho nhung pixel chua dat nguong phơi sáng
21         enhanced_image = enhanced_image + mask * a * (torch.pow(
enhanced_image, 2) - enhanced_image) * -1
22
23     return enhanced_image, x_r

```

Listing 9: Mã nguồn `forward` tối ưu hóa bằng Mặt nạ Thích ứng và Dừng sớm

Tài liệu

- [1] Guo, C., Li, C., Guo, J., Loy, C. C., Hou, J., Kwong, S., & Cong, R. (2020). Zero-reference deep curve estimation for low-light image enhancement. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 1780-1789).